

HelixQA

HelixQA

Anti-Bluff-QA-Orchestrierung - autonome, plattformübergreifende Sitzungen, bei denen jeder BESTANDEN-Nachweis erfasste Belege dafür liefert, dass ein echter Nutzer die Funktion tatsächlich verwenden kann.

Zusammenfassung

HelixQA ist ein Anti-Bluff-QA-Orchestrierungsframework für plattformübergreifendes Testen (Android, Android TV, Web, Desktop), das YAML-Testbanken, Echtzeit-Absturzerkennung, schrittweise Beweiserfassung sowie LLM-gestützte autonome QA-Sitzungen mit Computer Vision kombiniert, um nachzuweisen, dass Funktionen tatsächlich durchgängig funktionieren. Es ist der in Constitution vorgeschriebene QA-Testtyp (§11.4.169).

Kurzbeschreibung

Ein Anti-Bluff-QA-Orchestrator (Go), der geschriebene Testbanken und vollautonome, von LLM und Computer Vision gesteuerte QA-Sitzungen über verschiedene Plattformen hinweg ausführt - Abstürze erkennt, jeden Schritt anhand erfasster Beweise (Screenshots, Logcat, Video, Stack Traces) validiert und automatisch mit umfangreichen Belegen versehene Tickets für die AI-Fehlerbehebungs-Pipelines generiert.

Ausführliche Beschreibung

HelixQA ist ein Go-Framework, dessen einziges, kompromissloses Designprinzip die operative Regel §11.4 von Constitution ist: Die Messlatte für die Auslieferung lautet nicht „Tests bestehen“, sondern „Nutzer können die Funktion verwenden“. Daher muss jeder BESTANDEN-Nachweis positive Laufzeitbeweise enthalten, die während der Ausführung erfasst wurden - kein Beweis, kein

Grün, keine Ausnahmen. Es führt zwei sich ergänzende Modi aus, die gemeinsam sowohl das Skriptbasierte als auch das Unbekannte abdecken.

Erstens: **Geschriebene Testbanken** – YAML-Suiten mit TC-xxx-Testfällen, die Plattformausrüstung, Priorisierung, geordnete Schritte (Name/Aktion/Erwartung), Tags und Dokumentationsverweise umfassen. Diese werden mit schrittweiser Validierung, Echtzeit-Absturz-/ANR-Erkennung (ADB für Android, Prozessüberwachung für Web/Desktop), zentralisierter Beweiserfassung und automatisch generierten Markdown-Tickets ausgeführt, die bereits für die nachgelagerten AI-Fehlerbehebungs-Pipelines aufbereitet sind.

Zweitens: Eine **vollständig autonome QA-Sitzung**, bei der die App an LLM-gestützte Agenten und Computer Vision übergeben wird, die sie unbeaufsichtigt in vier disziplinierten Phasen steuern: Setup (Auswahl der LLMs, Erstellung einer Feature-Map aus Projektdokumenten, Start der CLI-Agenten, Initialisierung der Vision-Engine), dokumentenbasierte Überprüfung, die jedes dokumentierte Feature durchläuft, neugiergetriebene Erkundung, die gezielt Randfälle und undokumentiertes Verhalten testet, sowie abschließende Berichterstellung und Bereinigung in Markdown/HTML/JSON, wobei jeder Befund mit videogestempelten Beweisen verknüpft wird.

Entscheidend ist, dass das Framework seine eigenen Ergebnisse nicht selbst bewertet: Es integriert vier externe Go-Submodule (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) und nutzt die gemeinsame challenges- und containers-Infrastruktur. Dadurch ist die Komponente, die die App steuert, nicht identisch mit der, die bewertet, ob sie funktioniert hat. Die eigene Testsuite des Frameworks unterliegt exakt denselben Anforderungen, die es anderen auferlegt – überprüft durch make anti-bluff (statische Analyse + Verhaltensanker-Manifest + Mutationsratchet) und eine 8-phasige Orchestrator-Challenge mit integrierter §1.1-Mutation. Eine 15-zeilige Testtyp-Abdeckungsmatrix verknüpft jede beworbene Fähigkeit mit einem konkreten ausführbaren Asset und einer spezifischen Form der Beweiserfassung – sodass die Aussagen des Frameworks über sich selbst genauso beweisgebunden sind wie die Urteile, die es über die getesteten Produkte fällt.

Warum wir es entwickelt haben

Konventionelle QA gibt grünes Licht für „die Assertion ist bestanden“ – genau so schlüpft die von Constitution als *Bluff* bezeichnete Fehlerklasse durch: Ein Feature wird als funktionierend gemeldet, obwohl es für den echten Nutzer defekt ist. HelixQA wurde genau dafür entwickelt, um dies für die QA unmöglich zu machen: Es verweigert die Bewertung als

BESTANDEN, solange keine physischen Beweise (Screenshot, Logcat, Video, Stacktrace, Bericht) unter realer Ausführung erfasst wurden, und behandelt eine grüne Zusammenfassungszeile ohne solche Beweise als kritischen Defekt, der einem fehlenden Feature gleichkommt. Zudem löst es das Arbeitsproblem – umfassende manuelle QA über viele Plattformen skaliert nicht – indem es die Sessions vollständig autonom ablaufen lässt.

Warum es ein Game-Changer ist

Es vereint zwei Dinge, die fast nie in einem einzigen Tool zusammenfinden: rigorose, beweisgestützte QA-Gate-Kontrollen und autonome, selbstgesteuerte Exploration. Ein LLM-Plus-Vision-Agent öffnet die *tatsächliche* App, überprüft jedes dokumentierte Feature, spürt undokumentierte Bugs auf, für die niemand einen Test geschrieben hat, *und* erzeugt dabei eine gerichtsverwertbare Beweiskette – sodass „wir haben es getestet“ durch „hier ist das Video, hier der Logcat, hier das Ticket“ ersetzt wird. Und weil es sich um das von Constitution benannte QA-Submodul handelt, verbessert seine Einführung nicht nur die QA-Ehrlichkeit eines einzelnen Teams – sie hebt den Qualitätsstandard für jedes Produkt der Familie in einem einzigen Schritt an.

Was innovativ ist

- **Anti-Bluff-Beweisvertrag** – Jede BESTANDEN-Bewertung ist an erfasste Laufzeitbeweise gebunden; eine grüne CI-Zeile gilt als notwendig, aber niemals als ausreichend, und eine grüne Zusammenfassung ohne Beweise wird als kritischer Defekt eingestuft.
- **Autonome dokumenten- und neugiergetriebene Exploration** – Es überprüft jedes dokumentierte Feature *und* geht dann abseits des Skripts auf Erkundungstour, um die Randfälle zu testen, auf die echte Nutzer stoßen (leere Eingaben, schnelle Interaktionen, undokumentierte Pfade), die kein manuell geschriebener Testsuite vorhergesehen hat.
- **Vision-Orakel** – GoCV-Mechanikvision plus der LLM Vision API *sieht* buchstäblich die laufende UI auf dem Bildschirm und erkennt visuell fehlerhafte Zustände, die Token- und Property-basierte Assertions einfach übergehen.
- **Struktur- statt Prosa-Testbanken** – Bank-Strings beschreiben die Struktur und generieren zur Laufzeit LLM-Frageaufforderungen (CONST-046), sodass eine einzige Bank über alle Lokalisierungen hinweg funktioniert, statt beim Übersetzen der UI-Texte zu zerbrechen.
- **Tickets für AI-Fix-Pipelines** – Automatisch generierte Markdown-Issues kommen mit dem vollständigen Beweispaket

und sind bereit, direkt an einen nachgelagerten Reparatur-Agenten übergeben zu werden – ohne menschliche Triage.

Wie es in allen Produkten eingesetzt wird (die Fähigkeiten, die es verleiht)

Als **verpflichtender Qualitätsbaustein** (Constitution §11.4.169 benennt das `helix_qa`-Submodul als eine der vorgeschriebenen Testarten) verleiht HelixQA jedem Produkt der Familie dieselben Fähigkeiten:

- **Autonome QA-Sessions:** Ein einziger Befehl `helixqa autonomous --project ... --platforms android,desktop,web` setzt einen LLM-Plus-Vision-Agenten frei, der reale Apps unbeaufsichtigt bis zu einem Abdeckungsziel steuert und dabei Berichte, Tickets und Videos ohne menschliches Zutun erzeugt.
- **Testbanken / Suites:** YAML-Banken (Runde 219 mit Mindeststandard ≥ 30), plattformspezifisch, priorisiert und zeilenweise bis zu den Dokumenten zurückverfolgbar, die sie überprüfen.
- **Erfasste Beweise:** Screenshots, Logcat, Video, Stacktraces und eine vollständige Timeline – zentralisiert und mit jedem Bericht verknüpft, sodass jedes Urteil nachträglich nachvollzogen und geprüft werden kann.
- **Unabhängige Bewertungen (§11.4.141 Unabhängigkeitsprinzip):** Sein LLM-gestützter `issuedetector` und das Vision-Orakel beurteilen das Verhalten der laufenden App unabhängig vom steuernden Agenten und schließen damit strukturell den klassischen Fehler aus, bei dem ein System seine eigene Arbeit als korrekt markiert.
- **Gate + Mutationsratchet:** `make qa-all / make anti-bluff` und `challenges/scripts/helixqa_orchestrator_challenge.sh` (8 Phasen, integrierte §1.1-Mutation) beweisen kontinuierlich die Ehrlichkeit von HelixQA – und es gibt bewusst keine `--skip-helixqa`-Hintertür, um die Disziplin unter Zeitdruck auszuschalten.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Falsch-positive Ergebnisse in der QS selbst vermeiden** – das Tool, das Bluffs aufdeckt, darf nicht selbst zum Bluff werden → jeder Schritt wird anhand erfasster Beweise validiert, ein BESTANDEN ohne Belege wird als Fehler gewertet statt als Erfolg, und ein verhaltensverankertes Manifest verknüpft jede beworbene Funktion mit einem ausführbaren Test (CONST-035), sodass keine Funktion

behauptet werden kann, ohne dass sie tatsächlich geprüft wird.

- **Heterogene Plattformen mit einem einzigen Steuerungskern ansteuern** – Android, Android TV, Web und Desktop haben kein gemeinsames Eingabemodell → ein einziges navigator-Paket abstrahiert über plattformspezifische ActionExecutors (ADB, Playwright, X11) und plattformspezifische Absturz-Detektoren (Android/Web/Desktop), sodass die Orchestrierungslogik nur einmal geschrieben werden muss und die Plattformunterschiede an den Rändern bleiben.
- **Autonome Agenten nutzbringend statt chaotisch einsetzen** – ein unüberwachter LLM, der sich frei in einer App bewegt, kann endlos umherirren → LLMsVerifier bewertet und wählt die passenden Modelle aus, LLMOrchestrator verwaltet die headless CLI-Agenten (opencode, claude-code, gemini, junie, qwen-code), DocProcessor erstellt die Feature-Map, die der Exploration ein Ziel gibt, und VisionEngine verankert jede Entscheidung in den tatsächlichen Pixeln auf dem Bildschirm statt in der Vorstellung des Modells.
- **Lokalisierungssichere Testbanken** – eine Suite, die englische UI-Texte hartcodiert, scheitert in fünfzehn Sprachen → Testbanken beschreiben nur die Struktur, und der nutzerseitige Prompt-Text wird zur Laufzeit über LLM/Ressourcen geladen (CONST-046), sodass dieselbe Testbank dasselbe Verhalten unabhängig von der Sprache prüft.
- **Beweisen, dass die Kontrollmechanismen keine Scheinlösungen sind** – ein Anti-Bluff-Gate, das selbst nicht scheitern kann, ist der ultimative Bluff → gepaarte §1.1-Mutationen entfernen die Beweiserfassung oder Anti-Bluff-Assertion eines Typs und erzwingen, dass das Gate FEHLERHAFT reagiert, und ein Mutations-Ratschenmechanismus verhindert, dass diese Garantie im Laufe der Zeit unbemerkt untergraben wird.

Technologie-Stack

- **Go 1.24+ Orchestrator** – *Warum*: QS muss überall laufen, wo die Produkte eingesetzt werden, daher ist ein einziger statisch verlinkter, schneller und portabler Binärcode einer ressourcenintensiven Laufzeitumgebung überlegen; *Wie*: ein cmd/helixqa CLI mit kombinierbaren Unterbefehlen `run / list / report / autonomous / version`.
- **YAML-Testbanken (pkg/testbank)** – *Warum*: Testsuites sollten deklarativ und lesbar sein, von Menschen bearbeitbar, ohne Go anzufassen; *Wie*: `version/name/test_cases[]` mit `id`, `category`, `priority`, `platforms`, geordneten `steps[]` und `documentation_refs[]` für die Rückverfolgbarkeit zu den Feature-Dokumenten.

- **Absturz-/ANR-Detektoren (pkg/detector)** – *Warum:* Die schwerwiegendsten Fehler sind die, die live während der Interaktion auftreten, nicht in einer nachträglichen Assertion; *Wie:* ADB (pidof/logcat/screencap) für Android und pgrep für Web/Desktop, die den Prozess überwachen, während der Test ihn steuert.
- **Beweiserfassung (pkg/evidence, pkg/session)** – *Warum:* Der Anti-Bluff-Vertrag ist nur dann real, wenn jeder BESTANDEN durch physische Beweise untermauert wird; *Wie:* Screenshots, Logcat, Videos und Stacktraces werden in einer SessionRecorder-Zeitleiste erfasst, auf die jeder Bericht verweist.
- **Autonome Sitzung (pkg/autonomous, pkg/navigator, pkg/issuedetector)** – *Warum:* Manuelle QS über vier Plattformen hinweg skaliert nicht, daher muss die Exploration selbstgesteuert ablaufen; *Wie:* ein 4-Phasen-SessionCoordinator plus ActionExecutors (ADB/Playwright/X11) und LLM-Fehlererkennung, die visuelle, UX-, Barrierefreiheits- und funktionale Mängel abdeckt.
- **Externe Submodule** – *Warum:* Wiederverwendung und Entkopplung (CONST-051) sowie – entscheidend – die Trennung des Navigators vom Bewertungssystem; *Wie:* LLMsVerifier (Modellbewertung), LLMOrchestrator (headless CLI-Agenten), VisionEngine (GoCV + LLM Vision), DocProcessor (Feature-Map/Abdeckung), jeweils als eigenständige Komponente.
- **Anti-Bluff-Gates + Mutations-Ratschenmechanismus** – *Warum:* Um HelixQA auf genau die §1.1-Vereinbarung zu verpflichten, die es von allem anderen einfordert; *Wie:* ein make anti-bluff-Scan plus ein verhaltensverankertes Manifest und ein Mutations-Ratschenmechanismus, mit helixqa_orchestrator_challenge.sh als 8-Phasen-End-to-End-Validator.
- **15-Zeilen-Abdeckungsmatrix (docs/test-coverage.md)** – *Warum:* CONST-050(B) verlangt einen geschlossenen, vollständig erfassten Testtypensatz ohne Lücken; *Wie:* Jede Zeile ist an ein konkretes ausführbares Asset und eine spezifische Beweiserfassungsform gebunden, sodass die Abdeckung eine überprüfbare Tatsache ist und nicht nur eine Behauptung.

Status- & Ehrlichkeitshinweise

- **Status: Beta.** Aktive Entwicklung (README-Statusbanner Runde 219). Unterliegt dem eigenen Anti-Bluff-Standard.
- **Lizenz: Apache-2.0.** Installation: `go install digital.vasic.helixqa/cmd/helixqa@latest`.

Prioritätsstufe: Helix-primary — ein verbindlicher Qualitäts-/ Anti-Bluff-Pfeiler, der im Rahmen der Helix-Familie sicherstellt, dass Funktionen tatsächlich funktionieren.